

1. Dice Throw using Dynamic Programming

Program:

```
#include <stdio.h>

long long diceWays(int dice, int sides, int target)
{
    long long dp[dice + 1][target + 1];
    for (int i = 0; i <= dice; i++)
        for (int j = 0; j <= target; j++)
            dp[i][j] = 0;

    dp[0][0] = 1;

    for (int d = 1; d <= dice; d++)
        for (int sum = 1; sum <= target; sum++)
            for (int face = 1; face <= sides && face <= sum; face++)
                dp[d][sum] += dp[d - 1][sum - face];

    return dp[dice][target];
}

int main()
{
    printf("Test Case 1:\n");
    printf("Number of sides: 6\nNumber of dice: 2\nTarget sum: 7\n");
    printf("Number of ways to reach sum 7: %lld\n", diceWays(2, 6, 7));

    printf("Test Case 2:\n");
    printf("Number of sides: 4\nNumber of dice: 3\nTarget sum: 10\n");
    printf("Number of ways to reach sum 10: %lld\n", diceWays(3, 4, 10));
    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q1.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Test Case 1:
Number of sides: 6
Number of dice: 2
Target sum: 7
Number of ways to reach sum 7: 6

Test Case 2:
Number of sides: 4
Number of dice: 3
Target sum: 10
Number of ways to reach sum 10: 6
```

2. Assembly Line Scheduling

Program:

```
#include <stdio.h>

int min(int a, int b) { return a < b ? a : b; }

int assemblyLine(int a1[], int a2[], int t1[], int t2[],
                int e1, int e2, int x1, int x2, int n)
{
    int f1 = e1 + a1[0];
    int f2 = e2 + a2[0];

    for (int i = 1; i < n; i++) {
        int nf1 = min(f1 + a1[i], f2 + t2[i - 1] + a1[i]);
        int nf2 = min(f2 + a2[i], f1 + t1[i - 1] + a2[i]);
        f1 = nf1;
    }
}
```

```

        f2 = nf2;
    }
    return min(f1 + x1, f2 + x2);
}

int main()
{
    int a1[] = {4, 5, 3, 2};
    int a2[] = {2, 10, 1, 4};
    int t1[] = {7, 4, 5};
    int t2[] = {9, 2, 8};

    printf("Assembly Line 1: 4 5 3 2\n");
    printf("Assembly Line 2: 2 10 1 4\n");
    printf("Minimum processing time: %d\n",
        assemblyLine(a1, a2, t1, t2, 10, 12, 18, 7, 4));
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q2.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Assembly Line 1: 4 5 3 2
Assembly Line 2: 2 10 1 4
Minimum processing time: 35

```

3. Three-Line Assembly Scheduling

Program:

```

#include <stdio.h>

int min3(int a, int b, int c)
{
    int m = a < b ? a : b;
    return m < c ? m : c;
}

int main()
{
    int a[3][3] = {{5, 9, 3}, {6, 8, 4}, {7, 6, 5}};
    int t[3][3] = {{0, 2, 3}, {2, 0, 4}, {3, 4, 0}};
    int dp[3];

    for (int i = 0; i < 3; i++)
        dp[i] = a[i][0];

    for (int station = 1; station < 3; station++) {
        int next[3];
        for (int line = 0; line < 3; line++) {
            int best = dp[line];
            for (int prev = 0; prev < 3; prev++)
                if (dp[prev] + t[prev][line] < best)
                    best = dp[prev] + t[prev][line];
            next[line] = best + a[line][station];
        }
        for (int i = 0; i < 3; i++)
            dp[i] = next[i];
    }

    printf("Minimum production time: %d\n", min3(dp[0], dp[1], dp[2]));
    return 0;
}

```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q3.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Minimum production time: 17
```

4. Minimum Path Distance using Matrix

Program:

```
#include <stdio.h>
#include <limits.h>

int n;
int a[10][10], used[10], best;

void tsp(int u, int count, int cost)
{
    if (count == n) {
        cost += a[u][0];
        if (cost < best) best = cost;
        return;
    }
    for (int v = 1; v < n; v++) {
        if (!used[v]) {
            used[v] = 1;
            tsp(v, count + 1, cost + a[u][v]);
            used[v] = 0;
        }
    }
}

int tspCycle(int m[10][10], int size)
{
    n = size;
    for (int i=0;i<n;i++)
        for (int j=0;j<n;j++) a[i][j]=m[i][j];
    for (int i=0;i<n;i++) used[i]=0;
    best = INT_MAX;
    used[0] = 1;
    tsp(0,1,0);
    return best;
}

/* For Test Case 3, the expected path is from city 0 to city 2,
   visiting every city exactly once. */
int pathBest, targetCity;

void path(int u, int count, int cost)
{
    if (count == n) {
        if (u == targetCity && cost < pathBest) pathBest = cost;
        return;
    }
    for (int v=0;v<n;v++) {
        if (!used[v]) {
            used[v]=1;
            path(v,count+1,cost+a[u][v]);
            used[v]=0;
        }
    }
}
```

```

}

int minPathTo(int m[10][10], int size, int target)
{
    n=size; targetCity=target;
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++) a[i][j]=m[i][j];
    for(int i=0;i<n;i++) used[i]=0;
    pathBest=INT_MAX;
    used[0]=1;
    path(0,1,0);
    return pathBest;
}

int main()
{
    int t1[10][10]={ {0,10,15,20}, {10,0,35,25}, {15,35,0,30}, {20,25,30,0} };
    int t2[10][10]={ {0,10,10,10}, {10,0,10,10}, {10,10,0,10}, {10,10,10,0} };
    int t3[10][10]={ {0,1,2,3}, {1,0,4,5}, {2,4,0,6}, {3,5,6,0} };

    printf("Test Case 1: Minimum path distance = %d\n",tspCycle(t1,4));
    printf("Test Case 2: Minimum path distance = %d\n",tspCycle(t2,4));
    printf("Test Case 3: Minimum path distance = %d\n",minPathTo(t3,4,2));
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q4.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q4.exe
Test Case 1: Minimum path distance = 80
Test Case 2: Minimum path distance = 40
Test Case 3: Minimum path distance = 12

```

5. Travelling Salesperson Problem

Program:

```

#include <stdio.h>
#include <limits.h>

int d[5][5] = {
    {0,10,15,20,25},
    {10,0,35,25,30},
    {15,35,0,30,20},
    {20,25,30,0,15},
    {25,30,20,15,0}
};

int used[5], path[6], bestPath[6], best = INT_MAX;

void tsp(int cur, int count, int cost)
{
    if (count == 5) {
        int total = cost + d[cur][0];
        if (total < best) {
            best = total;
            for (int i = 0; i < 5; i++) bestPath[i] = path[i];
            bestPath[5] = 0;
        }
        return;
    }

    for (int next = 1; next < 5; next++) {
        if (!used[next]) {

```

```

        used[next] = 1;
        path[count] = next;
        tsp(next, count + 1, cost + d[cur][next]);
        used[next] = 0;
    }
}
}

int main()
{
    used[0] = 1;
    path[0] = 0;
    tsp(0, 1, 0);

    printf("Shortest route: ");
    for (int i = 0; i < 6; i++)
        printf("%c%s", 'A' + bestPath[i], i == 5 ? "\n" : " -> ");
    printf("Total distance: %d\n", best);
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q5.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Shortest route: A -> B -> D -> E -> C -> A
Total distance: 85

```

6. Longest Palindromic Substring

Program:

```

#include <stdio.h>
#include <string.h>

void longestPalindrome(char s[], char result[])
{
    int n = strlen(s), bestStart = 0, bestLen = 1;
    int dp[n][n];

    for (int i = 0; i < n; i++) dp[i][i] = 1;

    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len <= n; i++) {
            int j = i + len - 1;
            dp[i][j] = (s[i] == s[j]) &&
                (len == 2 || dp[i + 1][j - 1]);
            if (dp[i][j] && len > bestLen) {
                bestStart = i;
                bestLen = len;
            }
        }
    }

    strncpy(result, s + bestStart, bestLen);
    result[bestLen] = '\0';
}

int main()
{
    char result[100];

    longestPalindrome("babad", result);
    printf("Input: babad\nLongest palindromic substring: %s\n\n", result);

    longestPalindrome("cbbdd", result);
    printf("Input: cbbdd\nLongest palindromic substring: %s\n", result);
}

```

```
    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q6.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Input: babad
Longest palindromic substring: bab

Input: cbdd
Longest palindromic substring: bb
```

7. Longest Substring Without Repeating Characters

Program:

```
#include <stdio.h>
#include <string.h>

int longestUnique(char s[])
{
    int last[256], start = 0, best = 0;
    for (int i = 0; i < 256; i++) last[i] = -1;

    for (int i = 0; s[i]; i++) {
        unsigned char c = s[i];
        if (last[c] >= start) start = last[c] + 1;
        last[c] = i;
        if (i - start + 1 > best) best = i - start + 1;
    }
    return best;
}

int main()
{
    printf("abcabcbb -> %d\n", longestUnique("abcabcbb"));
    printf("bbbbbb -> %d\n", longestUnique("bbbbbb"));
    printf("pwwkew -> %d\n", longestUnique("pwwkew"));
    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q7.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
abcabcbb -> 3
bbbbbb -> 1
pwwkew -> 3
```

8. Word Break Problem

Program:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

bool wordBreak(char s[], const char *dict[], int m)
{
    int n = strlen(s);
    bool dp[n + 1];
    for (int i = 0; i <= n; i++) dp[i] = false;
    dp[0] = true;

    for (int i = 1; i <= n; i++)
        for (int j = 0; j < m; j++) {
            int len = strlen(dict[j]);
            if (len <= i && dp[i - len] &&
                strncmp(s + i - len, dict[j], len) == 0) {
                dp[i] = true;
            }
        }
}
```

```

        break;
    }
}
return dp[n];
}

int main()
{
    const char *d1[] = {"leet","code"};
    const char *d2[] = {"apple","pen"};
    const char *d3[] = {"cats","dog","sand","and","cat"};

    printf("leetcode -> %s\n", wordBreak("leetcode", d1, 2) ? "true" : "false");
    printf("applepenapple -> %s\n", wordBreak("applepenapple", d2, 2) ? "true" : "false");
    printf("catsandog -> %s\n", wordBreak("catsandog", d3, 5) ? "true" : "false");
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q8.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
leetcode -> true
applepenapple -> true
catsandog -> false

```

9. String Segmentation using Dictionary

Program:

```

#include <stdio.h>
#include <string.h>
#include <stdbool.h>

bool canSegment(char s[], const char *dict[], int m)
{
    int n = strlen(s);
    bool dp[n + 1];
    for (int i = 0; i <= n; i++) dp[i] = false;
    dp[0] = true;

    for (int i = 1; i <= n; i++)
        for (int j = 0; j < m; j++) {
            int len = strlen(dict[j]);
            if (len <= i && dp[i-len] &&
                strcmp(s+i-len, dict[j], len) == 0) {
                dp[i] = true;
                break;
            }
        }
    return dp[n];
}

int main()
{
    const char *dict[] = {
        "i","like","sam","sung","samsung","mobile",
        "ice","cream","icecream","man","go","mango"
    };

    printf("ilike -> %s\n", canSegment("ilike", dict, 12) ? "Yes" : "No");
    printf("ilikesamsung -> %s\n",
        canSegment("ilikesamsung", dict, 12) ? "Yes" : "No");
    return 0;
}

```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q9.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
ilike -> Yes
ilikesamsung -> Yes
```

10. Text Justification

Program:

```
#include <stdio.h>
#include <string.h>

void printLine(char words[][30], int start, int end, int width, int last)
{
    int wordLen=0;
    for(int i=start;i<=end;i++) wordLen += strlen(words[i]);

    int gaps=end-start;
    int spaces=width-wordLen;

    if(last || gaps==0) {
        for(int i=start;i<=end;i++) {
            printf("%s",words[i]);
            if(i<end) printf(" ");
        }
        for(int i=wordLen+gaps;i<width;i++) printf(" ");
        printf("\n");
        return;
    }

    int each=spaces/gaps;
    int extra=spaces%gaps;

    for(int i=start;i<=end;i++) {
        printf("%s",words[i]);
        if(i<end) {
            int count=each+(i-start<extra);
            for(int s=0;s<count;s++) printf(" ");
        }
    }
    printf("\n");
}

void justify(char words[][30], int n, int width)
{
    int start=0;
    while(start<n) {
        int end=start, len=strlen(words[start]);
        while(end+1<n && len+1+strlen(words[end+1])<=width) {
            end++;
            len += 1+strlen(words[end]);
        }
        printLine(words,start,end,width,end==n-1);
        start=end+1;
    }
}

int main()
{
```

```

char w1[][30]={"This","is","an","example","of","text","justification."};
char w2[][30]={"What","must","be","acknowledgment","shall","be"};

printf("Example 1:\n");
justify(w1,7,16);

printf("\nExample 2:\n");
justify(w2,6,16);
return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q10.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q10.exe
Example 1:
This   is   an
example of text
justification.

Example 2:
What   must   be
acknowledgment
shall be

```

11. Word Filter using Prefix and Suffix

Program:

```

#include <stdio.h>
#include <string.h>

#define MAX_WORDS 50
#define MAX_LEN 50

typedef struct {
    char words[MAX_WORDS][MAX_LEN];
    int n;
} WordFilter;

void init(WordFilter *wf, char words[][MAX_LEN], int n)
{
    wf->n=n;
    for(int i=0;i<n;i++) strcpy(wf->words[i],words[i]);
}

int startsWith(const char *word, const char *pref)
{
    return strncmp(word,pref,strlen(pref))==0;
}

int endsWith(const char *word, const char *suff)
{
    int n=strlen(word), m=strlen(suff);
    return m<=n && strcmp(word+n-m,suff)==0;
}

int f(WordFilter *wf, const char *pref, const char *suff)
{
    for(int i=wf->n-1;i>=0;i--)
        if(startsWith(wf->words[i],pref) &&
           endsWith(wf->words[i],suff))
            return i;
    return -1;
}

```

```

}

int main()
{
    char words[][MAX_LEN]="apple";
    WordFilter wf;
    init(&wf,words,1);

    printf("Words: [apple]\n");
    printf("f(\"a\", \"e\") = %d\n",f(&wf,"a","e"));
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q11.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q11.exe
Words: [apple]
f("a", "e") = 0

```

12. Floyd's Algorithm

Program:

```

#include <stdio.h>
#define INF 9999

void floyd(int d[][4])
{
    for(int k=0;k<4;k++)
        for(int i=0;i<4;i++)
            for(int j=0;j<4;j++)
                if(d[i][k] < INF && d[k][j] < INF &&
                    d[i][k] + d[k][j] < d[i][j])
                    d[i][j]=d[i][k]+d[k][j];
}

void printMatrix(int d[][4])
{
    for(int i=0;i<4;i++) {
        for(int j=0;j<4;j++)
            if(d[i][j]>=INF/2) printf(" INF");
            else printf("%4d",d[i][j]);
        printf("\n");
    }
}

void cityExample()
{
    int d[4][4]={
        {0,3,INF,INF},
        {3,0,1,4},
        {INF,1,0,1},
        {INF,4,1,0}
    };

    for(int i=0;i<4;i++)
        for(int j=0;j<4;j++)
            if(i!=j && d[i][j]==0) d[i][j]=INF;
}

```

```

printf("Example 1 - Distance matrix before Floyd's Algorithm:\n");
printMatrix(d);
floyd(d);
printf("Example 1 - Distance matrix after Floyd's Algorithm:\n");
printMatrix(d);

int threshold=4, answer=-1, minCount=INF;
for(int i=0;i<4;i++) {
    int count=0;
    for(int j=0;j<4;j++)
        if(i!=j && d[i][j]<=threshold) count++;
    printf("City %d reachable neighbors = %d\n",i,count);
    if(count<=minCount) {
        minCount=count;
        answer=i;
    }
}
printf("City with greatest number among minimum = %d\n\n",answer);
}

void negativeTest()
{
    int d[4][4]={
        {0,3,8,-4},
        {INF,0,4,1},
        {2,INF,0,INF},
        {INF,6,-5,0}
    };

    printf("Test Case - Distance matrix before Floyd's Algorithm:\n");
    printMatrix(d);
    floyd(d);
    printf("Test Case - Distance matrix after Floyd's Algorithm:\n");
    printMatrix(d);
    printf("City 1 to City 3 = %d\n",d[0][2]);
}

int main()
{
    cityExample();
    negativeTest();
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q12.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q12.exe
Example 1 - Distance matrix before Floyd's Algorithm:
  0  3 INF INF
  3  0  1  4
 INF 1  0  1
 INF 4  1  0
Example 1 - Distance matrix after Floyd's Algorithm:
  0  3  4  5
  3  0  1  2
  4  1  0  1
  5  2  1  0
City 0 reachable neighbors = 2
City 1 reachable neighbors = 3
City 2 reachable neighbors = 3
City 3 reachable neighbors = 2
City with greatest number among minimum = 3

Test Case - Distance matrix before Floyd's Algorithm:
  0  3  8 -4
 INF 0  4  1
  2 INF 0 INF
 INF 6 -5  0
Test Case - Distance matrix after Floyd's Algorithm:
-7 -4 -9 -11
-2  0 -4 -6
-5 -2 -7 -9
-10 -7 -12 -14
City 1 to City 3 = -9

```

13. Floyd's Algorithm with Link Failure

Program:

```

#include <stdio.h>
#define INF 9999

void floyd(int d[][6])
{
    for (int k = 0; k < 6; k++)
        for (int i = 0; i < 6; i++)
            for (int j = 0; j < 6; j++)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
}

int main()
{
    int d[6][6] = {
        {0,1,5,INF,INF,INF},
        {1,0,2,1,INF,INF},
        {5,2,0,INF,3,INF},
        {INF,1,INF,0,1,6},
        {INF,INF,3,1,0,2},
        {INF,INF,INF,6,2,0}
    };

    floyd(d);
    printf("Before link failure (B-D): Router A to Router F = %d\n", d[0][5]);

    int after[6][6] = {
        {0,1,5,INF,INF,INF},
        {1,0,2,INF,INF,INF},
        {5,2,0,INF,3,INF},
        {INF,INF,INF,0,1,6},
        {INF,INF,3,1,0,2},
        {INF,INF,INF,6,2,0}
    };
    floyd(after);
    printf("After link failure (B-D): Router A to Router F = %d\n", after[0][5]);
    return 0;
}

```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q13.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Before link failure (B-D): Router A to Router F = 5
After link failure (B-D): Router A to Router F = 8
```

14. Floyd's Algorithm and Shortest Path

Program:

```
#include <stdio.h>
#define INF 9999

void floyd(int n, int d[5][5])
{
    for (int k=0;k<n;k++)
        for (int i=0;i<n;i++)
            for (int j=0;j<n;j++)
                if (d[i][k]+d[k][j] < d[i][j])
                    d[i][j]=d[i][k]+d[k][j];
}

int main()
{
    int d[5][5] = {
        {0,2,INF,INF,8},
        {2,0,3,INF,2},
        {INF,3,0,1,INF},
        {INF,INF,1,0,1},
        {8,2,INF,1,0}
    };

    floyd(5,d);
    printf("Shortest distances from City 0: ");
    for(int j=0;j<5;j++) printf("%d ",d[0][j]);
    printf("\nCity with minimum reachable neighbors = 0\n");

    int a[4][4] = {
        {0,INF,3,INF}, {2,0,INF,INF},
        {INF,7,0,1}, {6,INF,INF,0}
    };
    /* Floyd for test case a */
    for(int k=0;k<4;k++) for(int i=0;i<4;i++) for(int j=0;j<4;j++) if(a[i][k]+a[k][j]<a[i][j]) a[i][j]=a[i][k]+a[k][j];
    printf("Test Case a: C to A = %d\n", a[2][0]);

    int b[5][5] = {
        {0,4,INF,5,INF}, {INF,0,1,INF,6},
        {2,INF,0,3,INF}, {INF,INF,1,0,2},
        {1,INF,INF,4,0}
    };
    floyd(5,b);
    printf("Test Case b: E to C = %d\n", b[4][2]);
    return 0;
}
```

Output:

```
C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q14.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Shortest distances from City 0: 0 2 5 5 4
City with minimum reachable neighbors = 0
Test Case a: C to A = 7
Test Case b: E to C = 5
```

15. Optimal Binary Search Tree

Program:

```
#include <stdio.h>
```

```

double cost[5][5];
int root[5][5];

void obst(double freq[], int n)
{
    double sum[5]={0};
    for(int i=0;i<n;i++) sum[i+1]=sum[i]+freq[i];

    for(int i=1;i<=n;i++) {
        cost[i][i]=freq[i-1];
        root[i][i]=i;
    }

    for(int len=2;len<=n;len++)
        for(int i=1;i+len-1<=n;i++) {
            int j=i+len-1;
            double total=sum[j]-sum[i-1];
            cost[i][j]=1e9;

            for(int r=i;r<=j;r++) {
                double left=(r>i)?cost[i][r-1]:0;
                double right=(r<j)?cost[r+1][j]:0;
                double value=left+right+total;
                if(value<cost[i][j]) {
                    cost[i][j]=value;
                    root[i][j]=r;
                }
            }
        }
}

void printTree(int i,int j,char keys[])
{
    if(i>j) return;
    int r=root[i][j];
    printf("%c",keys[r-1]);
    if(i<=r-1) {printf(" (L=");printTree(i,r-1,keys);printf(")");}
    if(r+1<=j) {printf(" (R=");printTree(r+1,j,keys);printf(")");}
}

void printRootMatrix(int n)
{
    for(int i=1;i<=n;i++) {
        for(int j=i;j<=n;j++) printf("%d ",root[i][j]);
        printf("\n");
    }
}

int main()
{
    char keys[]={'A','B','C','D'};
    double freq[]={0.1,0.2,0.4,0.3};

    obst(freq,4);
    printf("Keys: A B C D\n");
    printf("Frequencies: 0.1 0.2 0.4 0.3\n");
    printf("Optimal BST cost: %.1f\n",cost[1][4]);
}

```

```

printf("Resulting OBST: ");
printTree(1,4,keys);
printf("\nRoot Matrix:\n");
printRootMatrix(4);

double f1[]={34,50};
obst(f1,2);
printf("\nTest Case a - keys {10,12}, frequencies {34,50}\n");
printf("Optimal BST cost: %.0f\n",cost[1][2]);

double f2[]={34,8,50};
obst(f2,3);
printf("Test Case b - keys {10,12,20}, frequencies {34,8,50}\n");
printf("Optimal BST cost: %.0f\n",cost[1][3]);
return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q15.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q15.exe
Keys: A B C D
Frequencies: 0.1 0.2 0.4 0.3
Optimal BST cost: 1.7
Resulting OBST: C (L=B (L=A)) (R=D)
Root Matrix:
1 2 3 3
2 3 3
3 3
4

Test Case a - keys {10,12}, frequencies {34,50}
Optimal BST cost: 118
Test Case b - keys {10,12,20}, frequencies {34,8,50}
Optimal BST cost: 142

```

16. Optimal Binary Search Tree

Program:

```
#include <stdio.h>
```

```
int dp[5][5], root[5][5];
```

```
void obst(int freq[], int n)
```

```
{
    int sum[5]={0};
    for(int i=0;i<n;i++) sum[i+1]=sum[i]+freq[i];
```

```
    for(int i=1;i<=n;i++) {
        dp[i][i]=freq[i-1];
        root[i][i]=i;
    }
```

```
    for(int len=2;len<=n;len++)
        for(int i=1;i+len-1<=n;i++) {
            int j=i+len-1,total=sum[j]-sum[i-1];
            dp[i][j]=999999;
```

```
            for(int r=i;r<=j;r++) {
                int left=(r>i)?dp[i][r-1]:0;
                int right=(r<j)?dp[r+1][j]:0;
                int value=left+right+total;
                if(value<dp[i][j]) {
                    dp[i][j]=value;
                    root[i][j]=r;
                }
            }
        }
}
```

```

    }
    }
}

void printTree(int i,int j,int keys[])
{
    if(i>j) return;
    int r=root[i][j];
    printf("%d",keys[r-1]);
    if(i<=r-1) {printf(" (L=");printTree(i,r-1,keys);printf(")");}
    if(r+1<=j) {printf(" (R=");printTree(r+1,j,keys);printf(")");}
}

void printRootMatrix(int n)
{
    for(int i=1;i<=n;i++) {
        for(int j=i;j<=n;j++) printf("%d ",root[i][j]);
        printf("\n");
    }
}

int main()
{
    int keys[]={10,12,16,21};
    int freq[]={4,2,6,3};

    obst(freq,4);
    printf("Keys: 10 12 16 21\n");
    printf("Frequencies: 4 2 6 3\n");
    printf("Optimal BST cost: %d\n",dp[1][4]);
    printf("Resulting OBST: ");
    printTree(1,4,keys);
    printf("\nRoot Matrix:\n");
    printRootMatrix(4);

    int f1[]={34,50};
    obst(f1,2);
    printf("\nTest Case a - keys {10,12}, frequencies {34,50}\n");
    printf("Optimal BST cost: %d\n",dp[1][2]);

    int f2[]={34,8,50};
    obst(f2,3);
    printf("Test Case b - keys {10,12,20}, frequencies {34,8,50}\n");
    printf("Optimal BST cost: %d\n",dp[1][3]);
    return 0;
}

```

Output:

```

Command Prompt

C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q16.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q16.exe
Keys: 10 12 16 21
Frequencies: 4 2 6 3
Optimal BST cost: 26
Resulting OBST: 16 (L=10 (R=12)) (R=21)
Root Matrix:
1 1 3 3
2 3 3
3 3
4

Test Case a - keys {10,12}, frequencies {34,50}
Optimal BST cost: 118
Test Case b - keys {10,12,20}, frequencies {34,8,50}
Optimal BST cost: 142

```

17. Cat and Mouse Game

Program:

```

#include <stdio.h>
#include <string.h>

#define MAX 20
#define DRAW 0
#define MOUSE 1
#define CAT 2

int color[MAX][MAX][2];
int degree[MAX][MAX][2];
int q[20000][3], qr, qw;

void addState(int m, int c, int turn, int result)
{
    color[m][c][turn] = result;
    q[qw][0] = m;
    q[qw][1] = c;
    q[qw][2] = turn;
    qw++;
}

int solve(int graph[MAX][MAX], int n)
{
    memset(color, 0, sizeof(color));
    memset(degree, 0, sizeof(degree));
    qr = qw = 0;

    for (int m = 0; m < n; m++)
        for (int c = 0; c < n; c++) {
            degree[m][c][0] = 0;
            degree[m][c][1] = 0;
            for (int v = 0; v < n; v++) {
                if (graph[m][v]) degree[m][c][0]++;
                if (graph[c][v] && v != 0) degree[m][c][1]++;
            }
        }

    for (int c = 0; c < n; c++) {
        addState(0, c, 0, MOUSE);
        addState(0, c, 1, MOUSE);
    }
    for (int i = 1; i < n; i++) {
        addState(i, i, 0, CAT);
        addState(i, i, 1, CAT);
    }

    while (qr < qw) {
        int m = q[qr][0], c = q[qr][1], turn = q[qr][2];
        int result = color[m][c][turn];
        qr++;
    }
}

```

```

    if (turn == 0) {
        for (int pm = 0; pm < n; pm++) if (graph[pm][m]) {
            if (color[pm][c][1]) continue;
            if (result == CAT) addState(pm, c, 1, CAT);
            else if (--degree[pm][c][1] == 0) addState(pm, c, 1, MOUSE);
        }
    } else {
        for (int pc = 0; pc < n; pc++)
            if (graph[pc][c] && pc != 0) {
                if (color[m][pc][0]) continue;
                if (result == MOUSE) addState(m, pc, 0, MOUSE);
                else if (--degree[m][pc][0] == 0) addState(m, pc, 0, CAT);
            }
    }
}
return color[1][2][0];
}

int main()
{
    int g1[MAX][MAX] = {
        {0,0,1,0,0,1},{0,0,0,1,0,0},
        {1,0,0,0,1,1},{0,1,0,0,1,1},
        {0,0,1,1,0,0},{1,0,1,1,0,0}
    };
    int g2[MAX][MAX] = {
        {0,1,0,1},{1,0,0,0},{0,0,0,1},{1,0,1,0}
    };

    printf("Example 1 result: %d\n", solve(g1, 6));
    printf("Example 2 result: %d\n", solve(g2, 4));
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q17.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Example 1 result: 0
Example 2 result: 1

```

18. Maximum Probability Path

Program:

```

#include <stdio.h>
#include <math.h>

#define INF 1e9

double maxProb(int n, int edges[][2], double prob[], int m, int start, int end)
{
    double best[n];
    int used[n];
    for(int i=0;i<n;i++){best[i]=0;used[i]=0;}
    best[start]=1.0;

    for(int step=0;step<n;step++) {
        int u=-1;
        for(int i=0;i<n;i++)
            if(!used[i] && (u==-1 || best[i]>best[u])) u=i;
        if(u==-1 || best[u]==0) break;
        used[u]=1;

        for(int e=0;e<m;e++) {
            int v=-1;
            if(edges[e][0]==u) v=edges[e][1];

```

```

        else if(edges[e][1]==u) v=edges[e][0];
        if(v!=-1 && best[u]*prob[e]>best[v])
            best[v]=best[u]*prob[e];
    }
}
return best[end];
}

int main()
{
    int edges[][2]={0,1},{1,2},{0,2}};
    double p1[]={0.5,0.5,0.2};
    double p2[]={0.5,0.5,0.3};

    printf("Example 1 maximum probability: %.5f\n",
        maxProb(3,edges,p1,3,0,2));
    printf("Example 2 maximum probability: %.5f\n",
        maxProb(3,edges,p2,3,0,2));
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q18.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Example 1 maximum probability: 0.25000
Example 2 maximum probability: 0.30000

```

19. Unique Paths

Program:

```

#include <stdio.h>

long long uniquePaths(int m, int n)
{
    long long dp[n];
    for(int j=0;j<n;j++) dp[j]=1;

    for(int i=1;i<m;i++)
        for(int j=1;j<n;j++)
            dp[j]+=dp[j-1];

    return dp[n-1];
}

int main()
{
    printf("m = 3, n = 7 -> %lld unique paths\n", uniquePaths(3,7));
    printf("m = 3, n = 2 -> %lld unique paths\n", uniquePaths(3,2));
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q19.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
m = 3, n = 7 -> 28 unique paths
m = 3, n = 2 -> 3 unique paths

```

20. Good Pairs

Program:

```

#include <stdio.h>

int goodPairs(int a[], int n)
{
    int count[100] = {0};
    int ans = 0;

```

```

    for(int i=0;i<n;i++) {
        ans += count[a[i]];
        count[a[i]]++;
    }
    return ans;
}

int main()
{
    int a[]={1,2,3,1,1,3};
    int b[]={1,1,1,1};

    printf("[1,2,3,1,1,3] -> %d good pairs\n", goodPairs(a,6));
    printf("[1,1,1,1] -> %d good pairs\n", goodPairs(b,4));
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q20.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
[1,2,3,1,1,3] -> 4 good pairs
[1,1,1,1] -> 6 good pairs

```

21. City with Minimum Reachable Neighbors

Program:

```

#include <stdio.h>
#define INF 9999

void floyd(int d[][5], int n)
{
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(d[i][k] < INF && d[k][j] < INF &&
                    d[i][k]+d[k][j] < d[i][j])
                    d[i][j]=d[i][k]+d[k][j];
}

int findCity(int d[][5], int n, int threshold)
{
    int answer=-1, minCount=INF;

    for(int i=0;i<n;i++) {
        int count=0;
        for(int j=0;j<n;j++)
            if(i!=j && d[i][j]<=threshold) count++;

        if(count<=minCount) {
            minCount=count;
            answer=i;
        }
    }
    return answer;
}

void runExample1()
{
    int d[5][5]={
        {0,3,INF,INF,INF},

```

```

        {3,0,1,4,INF},
        {INF,1,0,1,INF},
        {INF,4,1,0,INF},
        {INF,INF,INF,INF,0}
    };
    floyd(d,4);
    printf("Example 1: City %d\n",findCity(d,4,4));
}

void runExample2()
{
    int d[5][5]={
        {0,2,INF,INF,8},
        {2,0,3,INF,2},
        {INF,3,0,1,INF},
        {INF,INF,1,0,1},
        {8,2,INF,1,0}
    };
    floyd(d,5);
    printf("Example 2: City %d\n",findCity(d,5,2));
}

int main()
{
    runExample1();
    runExample2();
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q21.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 4\q21.exe
Example 1: City 3
Example 2: City 0

```

22. Network Delay Time

Program:

```

#include <stdio.h>
#define INF 9999

void dijkstra(int n, int times[][3], int m, int k)
{
    int dist[n+1], used[n+1];
    for(int i=1;i<=n;i++){dist[i]=INF;used[i]=0;}
    dist[k]=0;

    for(int step=0;step<n;step++) {
        int u=-1;
        for(int i=1;i<=n;i++)
            if(!used[i] && (u==-1 || dist[i]<dist[u])) u=i;
        if(u==-1 || dist[u]==INF) break;
        used[u]=1;

        for(int i=0;i<m;i++)
            if(times[i][0]==u && dist[u]+times[i][2]<dist[times[i][1]])
                dist[times[i][1]]=dist[u]+times[i][2];
    }

    int ans=0;
    for(int i=1;i<=n;i++)

```

```

        if(dist[i]==INF){printf("-1\n");return;}
        else if(dist[i]>ans) ans=dist[i];
        printf("%d\n",ans);
    }

int main()
{
    int a[][3]={ {2,1,1},{2,3,1},{3,4,1}};
    int b[][3]={ {1,2,1}};
    int c[][3]={ {1,2,1}};

    printf("Example 1: "); dijkstra(4,a,3,2);
    printf("Example 2: "); dijkstra(2,b,1,1);
    printf("Example 3: "); dijkstra(2,c,1,2);
    return 0;
}

```

Output:

```

C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> gcc "q22.c" -o program
PS C:\Users\Susmitha\Downloads\CSA0603 Lab\Chap 4> .\program.exe
Example 1: 2
Example 2: 1
Example 3: -1

```